FastAPI Sentiment Analysis App

Prerequisites

- Google Colab Account
- Required Libraries like FastAPI, Uvicorn, Nest-Asyncio, Pyngrok, Transformers, and Torch

Run below command to install dependencies

```
!pip install fastapi uvicorn nest-asyncio pyngrok transformers torch
```

```
Collecting fastapi
  Downloading fastapi-0.115.5-py3-none-any.whl.metadata (27 kB)
Collecting uvicorn
  Downloading uvicorn-0.32.0-py3-none-any.whl.metadata (6.6 kB)
Requirement already satisfied: nest-asyncio in /usr/local/lib/python3.10/dist-packages (1.6.0)
Collecting pyngrok
  Downloading pyngrok-7.2.1-py3-none-any.whl.metadata (8.3 kB)
Requirement already satisfied: transformers in /usr/local/lib/python3.10/dist-packages (4.46.2)
Requirement already satisfied: torch in /usr/local/lib/python3.10/dist-packages (2.5.0+cu121)
Collecting starlette<0.42.0,>=0.40.0 (from fastapi)
  Downloading starlette-0.41.2-py3-none-any.whl.metadata (6.0 kB)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,!=2.0.0,!=2.0.1,!=2.1.0,<3.0.0,>=1.7.4 in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.10/dist-packages (from fastapi) (4.12.2)
Requirement already satisfied: click>=7.0 in /usr/local/lib/python3.10/dist-packages (from uvicorn) (8.1.7)
Requirement already satisfied: h11>=0.8 in /usr/local/lib/python3.10/dist-packages (from uvicorn) (0.14.0)
Requirement already satisfied: PyYAML>=5.1 in /usr/local/lib/python3.10/dist-packages (from pyngrok) (6.0.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from transformers) (3.16.1)
Requirement already satisfied: huggingface-hub<1.0,>=0.23.2 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.26.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from transformers) (24.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.10/dist-packages (from transformers) (2024.9.11)
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from transformers) (2.32.3)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.4.5)
Requirement already satisfied: tokenizers<0.21,>=0.20 in /usr/local/lib/python3.10/dist-packages (from transformers) (0.20.3)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.10/dist-packages (from transformers) (4.66.6)
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from torch) (3.4.2)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.10/dist-packages (from torch) (3.1.4)
Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages (from torch) (2024.10.0)
Requirement already satisfied: sympy==1.13.1 in /usr/local/lib/python3.10/dist-packages (from torch) (1.13.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from sympy==1.13.1->torch) (1.3.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.10/dist-packages (from pydantic!=1.8,!=1.8.1,!=2.0.0
Requirement already satisfied: pydantic-core==2.23.4 in /usr/local/lib/python3.10/dist-packages (from pydantic!=1.8,!=1.8.1,!=2.0.0,
Requirement already satisfied: anyio<5,>=3.4.0 in /usr/local/lib/python3.10/dist-packages (from starlette<0.42.0,>=0.40.0->fastapi)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from jinja2->torch) (3.0.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->transformers) (2024.8.3
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.10/dist-packages (from anyio<5,>=3.4.0->starlette<0.42.0,>=0.4
Requirement already satisfied: exceptiongroup in /usr/local/lib/python3.10/dist-packages (from anyio<5,>=3.4.0->starlette<0.42.0,>=0
Downloading fastapi-0.115.5-py3-none-any.whl (94 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 94.9/94.9 kB 1.7 MB/s eta 0:00:00
Downloading uvicorn-0.32.0-py3-none-any.whl (63 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 63.7/63.7 kB 2.0 MB/s eta 0:00:00
Downloading pyngrok-7.2.1-py3-none-any.whl (22 kB)
Downloading starlette-0.41.2-py3-none-any.whl (73 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 73.3/73.3 kB 4.2 MB/s eta 0:00:00
Installing collected packages: uvicorn, pyngrok, starlette, fastapi
Successfully installed fastapi-0.115.5 pyngrok-7.2.1 starlette-0.41.2 uvicorn-0.32.0
```

```
!pip install pyngrok
```

```
Collecting pyngrok
  Downloading pyngrok-7.2.1-py3-none-any.whl.metadata (8.3 kB)
Requirement already satisfied: PyYAML>=5.1 in /usr/local/lib/python3.10/dist-packages (from pyngrok) (6.0.2)
Downloading pyngrok-7.2.1-py3-none-any.whl (22 kB)
Installing collected packages: pyngrok
Successfully installed pyngrok-7.2.1
```

```
pip install python-multipart
```

```
Collecting python-multipart
  Downloading python_multipart-0.0.17-py3-none-any.whl.metadata (1.8 kB)
Downloading python_multipart-0.0.17-py3-none-any.whl (24 kB)
Installing collected packages: python-multipart
```

```
Successfully installed python-multipart-0.0.17
```

```
File Structure

├── main.py                 # FastAPI application and routing
├── sentiment_model.py      # Model logic for sentiment prediction
└── templates/
    ├── index.html          # Home page with text input
    └── result.html         # Displays sentiment results
```

File Structure

├── main.py # FastAPI application and routing ├── sentiment_model.py # Model logic for sentiment prediction └── templates/ ├── index.html # Home page with text input └── result.html # Displays sentiment results

the FastAPI app directly in the notebook that we will save it to a main.py file and run it from there

```
# Write FastAPI code to main.py
with open("main.py", "w") as f:
```

this script writes the FastAPI code to a file named main.py and Opens the file main.py in write mode ("w"). The with statement ensures that the file is properly closed after writing.

```
f.write('...'):
```

Writes the provided code as a string to main.py.

Inside the Written Code:

```
from fastapi import FastAPI, Form:
```

Imports FastAPI for creating the web application and Form for parsing form data from HTTP requests.

```
from transformers import pipeline:
```

Imports pipeline from the transformers library, which provides easy-to-use pipelines for various NLP tasks.

```
from fastapi.responses import JSONResponse:
```

Imports JSONResponse for returning JSON responses from the API.

Initialize FastAPI App

```
app = FastAPI():
```

Creates an instance of the FastAPI application.

```
sentiment_analyzer = pipeline("sentiment-analysis"):
```

Initializes a sentiment analysis pipeline using the Hugging Face transformers library. This pipeline will be used to analyze the sentiment of the provided text.

Define API Routes

```
# Root route:
```

A comment indicating the root route of the API.

```
@app.get("/"):
```

Defines a GET endpoint at the root URL (/). When a GET request is made to this URL, the home function is called.

```
async def home()::
```

Defines an asynchronous function home that handles requests to the root URL.

```
return {"message": "Welcome to the Sentiment Analysis API!"}:
```

Returns a JSON response with a welcome message when the root URL is accessed.

```
# Predict route:
```

A comment indicating the predict route of the API.

```
@app.post("/predict/"):
```

Defines a POST endpoint at the URL /predict/. When a POST request is made to this URL, the predict function is called.

```
async def predict(text: str = Form(...)):
```

Defines an asynchronous function predict that handles POST requests to the /predict/ URL.

The text parameter is expected to be passed as form data.

```
result = sentiment_analyzer(text)[0]:
```

- Uses the sentiment_analyzer pipeline to analyze the sentiment of the provided text.
- The result is a list of predictions, so [0] is used to access the first prediction

```
return JSONResponse(content={"label": result["label"], "score": round(result["score"], 2)}):
```

Returns a JSON response with the prediction label and score rounded to two decimal places.

How It Connects to main.py

- It initializes FastAPI, creates a sentiment analysis pipeline, and sets up the API routes.
- It defines a root route (/) and a predict route (/predict/) for handling requests.
- The responses from the API are sent back as JSON, making it easy to integrate with other services or front-end applications.

main.py

```
from fastapi import FastAPI, Form
from transformers import pipeline
from fastapi.responses import JSONResponse
```

Imports: -> FastAPI: This is the main class for creating the FastAPI application.

-> Form: This is used to handle form data in POST requests.

-> pipeline: This function from the transformers library creates a pipeline for specific tasks, such as sentiment analysis.

-> JSONResponse: This is used to return responses in JSON format.

```
# Initialize FastAPI app
app = FastAPI()
sentiment_analyzer = pipeline("sentiment-analysis")
```

Initialize FastAPI app:

-> app = FastAPI(): Creates an instance of the FastAPI application.

-> sentiment_analyzer = pipeline("sentiment-analysis"): Initializes a sentiment analysis pipeline using a pre-trained model from the transformers library. This will be used to analyze the sentiment of the input text.

```
# Root route
@app.get("/")
async def home():
    return {"message": "Welcome to the Sentiment Analysis API!"}
```

Root route:

-> @app.get("/"): This decorator defines a GET endpoint at the root URL (/). When a GET request is made to this URL, the home function is called.

-> async def home(): Defines an asynchronous function named home that will handle the GET requests to the root URL.

-> return {"message": "Welcome to the Sentiment Analysis API!"}: Returns a JSON response with a welcome message.

```
# Predict route
@app.post("/predict/")
async def predict(text: str = Form(...)):
    result = sentiment_analyzer(text)[0]
    return JSONResponse(content={"label": result["label"], "score": round(result["score"], 2)})
```

Predict route:

-> @app.post("/predict/"): This decorator defines a POST endpoint at the URL /predict/. When a POST request is made to this URL, the predict function is called.

-> async def predict(text: str = Form(...)): Defines an asynchronous function named predict that will handle the POST requests to the /predict/ URL. The text parameter is expected to be passed as form data.

-> result = sentiment_analyzer(text)[0]: Uses the sentiment_analyzer pipeline to analyze the sentiment of the provided text. The result is a list of predictions, so [0] is used to access the first prediction.

-> return JSONResponse(content={"label": result["label"], "score": round(result["score"], 2)}): Returns a JSON response with the prediction label (e.g., "POSITIVE" or "NEGATIVE") and the confidence score rounded to two decimal places.

Setting Up Ngrok for Tunneling

```
from pyngrok import ngrok
```

This line imports the ngrok module from the pyngrok library, which allows you to create secure tunnels to your localhost, making your local web server accessible over the internet.

```
# Set your Ngrok authtoken
ngrok.set_auth_token("2onZtO59Q5MQUVfKJuAKrmynuZ2_53E2CoCeJhvNEkavBNWGW")  # Replace with your actual Ngrok authtoken
```

This line sets the authentication token for ngrok. The token is required to authenticate your account with ngrok and allow you to use their tunneling services. You should replace the placeholder token with your actual ngrok authtoken.

```
# Open a public URL for port 8000
public_url = ngrok.connect(8000)
```

This line creates a tunnel that exposes your local server running on port 8000 to the internet. The ngrok.connect(8000) function call returns a public URL that you can use to access your local server from anywhere.

```
print(f"Public URL: {public_url}")
```

This line prints the public URL generated by ngrok. You can use this URL to access your local server from any device with internet access.